APACHE KAFKA

Exactly-once Semantics are Possible: Here's How Kafka Does it

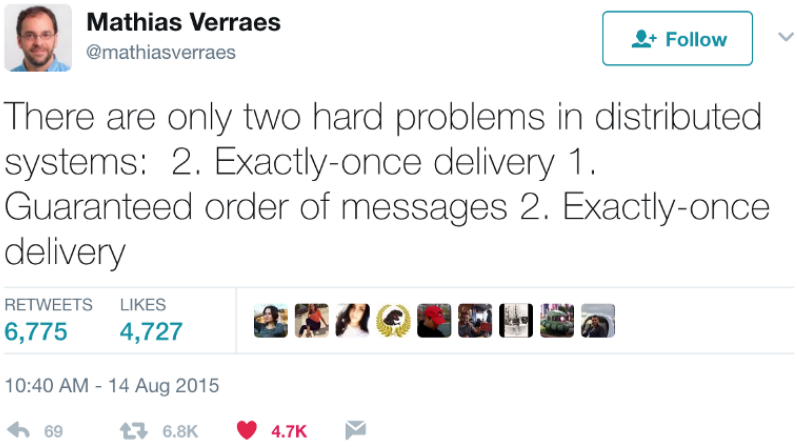
Author: Neha Narkhede

June 30, 2017

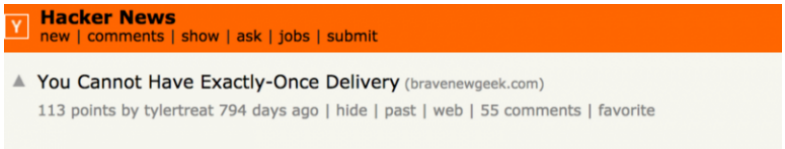
I'm thrilled that we have hit an exciting milestone the Kafka community has long been waiting for: we have introduced exactly-once semantics in Apache Kafka in the 0.11 release and Confluent Platform 3.3. In this post, I'd like to tell you what exactly-once semantics mean in Apache Kafka, why it is a hard problem, and how the new idempotence and transactions features in Kafka enable correct exactly-once stream processing using Kafka's Streams API.

Exactly-once is a really hard problem

Now, I know what some of you are thinking. Exactly-once delivery is impossible, it comes at too high a price to put it to practical use, or that I'm getting all this entirely wrong! You're not alone in thinking that. Some of my industry colleagues recognize that exactly-once delivery is one of the hardest problems to solve in distributed systems.



While some have outright said that exactly-once delivery is probably impossible!



Now, I don't deny that introducing exactly-once delivery semantics—and supporting exactly-once stream processing—is a truly hard problem to solve. But I've also witnessed smart distributed systems engineers at Confluent work diligently with the open-source community for over a year to solve this problem in Apache Kafka. So let's jump right in with an overview of messaging semantics.

Overview of messaging semantics

In a distributed system, the computers that make up the system can always fail independently of one another. In the case of Kafka, an individual broker can crash, or a network failure can happen while the producer is sending a message to a topic. Depending on the action the producer takes to handle such a failure, you can get different semantics:

- **At least once semantics**: if the producer receives an acknowledgement (ack) from the Kafka broker and acks=all, it means that the message has been written exactly once to the Kafka topic. However, if a producer ack times out or receives an error, it might retry sending the message assuming that the message was not written to the Kafka topic. If the broker had failed right before it sent the ack but after the message was successfully written to the Kafka topic, this retry leads to the message being written twice and hence delivered more than once to the end consumer. And everybody loves a cheerful giver, but this approach can lead to duplicated work and incorrect results.
- **At most once semantics**: if the producer does not retry when an ack times out or returns an error, then the message might end up not being written to the Kafka topic, and hence not delivered to the consumer. In most cases it will be, but in order to avoid the possibility of duplication, we accept that sometimes messages will not get through.
- **Exactly once semantics**: even if a producer retries sending a message, it leads to the message being delivered exactly once to the end consumer. Exactly-once semantics is the most desirable guarantee, but also a poorly understood one. This is because it requires a cooperation between the messaging system itself and the application producing and consuming the messages. For instance, if after consuming a message successfully you rewind your Kafka consumer to a previous offset, you will receive all the messages from that offset to the latest one, all over again. This shows why the messaging system and the client application must cooperate to make exactly-once semantics happen.

Failures that must be handled

To describe the challenges involved in supporting exactly-once delivery semantics, let's start with a simple example.

Suppose there is a single-process producer application that sends the message "Hello Kafka" to a single-partition Kafka topic called "EoS." Further suppose that a single-instance consumer application on the other end pulls data from the topic and prints the message. In the happy path where there are no failures, this works well, and the message "Hello Kafka" is written to the EoS topic partition only once. The consumer pulls the message, processes it, and commits the message offset to indicate that it has completed its processing. It will not receive it again, even if the consumer application fails and restarts.

However, we all know that we can't count on the happy path. At scale, even unlikely failure scenarios are things that end up happening all the time.

1. **A broker can fail**: Kafka is a highly available, persistent, durable system where every message written to a partition is persisted and replicated some number of times (we will call it *n*). As a result, Kafka can tolerate *n-1* broker failures, meaning that a partition is available as long as there is at least one broker available. Kafka's replication protocol guarantees that once a message has been written successfully to the leader replica, it will be replicated to all available replicas.
2. **The producer-to-broker RPC can fail:** Durability in Kafka depends on the producer receiving an ack from the broker. Failure to receive that ack does not necessarily mean that the request itself failed. The broker can crash after writing a message but before it sends an ack back to the producer. It can also crash before even writing the message to the topic. Since there is no way for the producer to know the nature of the failure, it is forced to assume that the message was not written successfully and to retry it. In some cases, this will cause the same message to be duplicated in the Kafka partition log, causing the end consumer to receive it more than once.
3. **The client can fail**: Exactly-once delivery must account for client failures as well. But how can we know that a client has actually failed and is not just temporarily partitioned from the brokers or undergoing an application pause? Being able to tell the difference between a permanent failure and a soft one is important; for correctness, the broker should discard messages sent by a zombie producer. Same is true for the consumer; once a new client instance has been started, it must be able to recover from whatever state the failed instance left behind and begin processing from a safe point. This means that consumed offsets must always be kept in sync with produced output.

Exactly-once semantics in Apache Kafka, explained

Prior to 0.11.x, Apache Kafka supported at-least once delivery semantics and in-order delivery per partition. As you can tell from the example above, that means producer retries can cause duplicate messages. In the new exactly-once semantics feature, we've strengthened Kafka's processing semantics in three different and interrelated ways.

Idempotence: Exactly-once in order semantics per partition

An *idempotent* operation is one which can be performed many times without causing a different effect than only being performed once. The producer send operation is now idempotent. In the event of an error that causes a producer retry, the same message—which is still sent by the producer multiple times—will only be written to the Kafka log on the broker once. For a single

partition, Idempotent producer sends remove the possibility of duplicate messages due to producer or broker errors. To turn on this feature and get exactly-once semantics per partition—meaning no duplicates, no data loss, and in-order semantics—configure your producer to set “enable.idempotence=true”.

How does this feature work? Under the covers it works in a way similar to TCP; each batch of messages sent to Kafka will contain a sequence number which the broker will use to dedupe any duplicate send. Unlike TCP, though—which provides guarantees only within a transient in-memory connection—this sequence number is persisted to the replicated log, so even if the leader fails, any broker that takes over will also know if a resend is a duplicate. The overhead of this mechanism is quite low: it’s just a few extra numeric fields with each batch of messages. As you will see later in this article, this feature add negligible performance overhead over the non-idempotent producer.

Transactions: Atomic writes across multiple partitions

Second, Kafka now supports atomic writes across multiple partitions through the new transactions API. This allows a producer to send a batch of messages to multiple partitions such that either all messages in the batch are eventually visible to any consumer or none are ever visible to consumers. This feature also allows you to commit your consumer offsets in the same transaction along with the data you have processed, thereby allowing end-to-end exactly-once semantics. Here’s an example code snippet to demonstrate the use of the transactions API—

```
producer.initTransactions();
try {
  producer.beginTransaction();
  producer.send(record1);
  producer.send(record2);
  producer.commitTransaction();
} catch(ProducerFencedException e) {
  producer.close();
} catch(KafkaException e) {
  producer.abortTransaction();
}
```

The code snippet above describes how you can use the new Producer APIs to send messages atomically to a set of topic partitions. It is worth noting that a Kafka topic partition might have some messages that are part of a transaction while others that are not.

So on the Consumer side, you have two options for reading transactional messages, expressed through the “isolation.level” consumer config:

1. `read_committed` : In addition to reading messages that are not part of a transaction, also be able to read ones that are, after the transaction is committed.
2. `read_uncommitted` : Read all messages in offset order without waiting for transactions to be committed. This option is similar to the current semantics of a Kafka consumer.

To use transactions, you need to configure the Consumer to use the right “isolation.level”, use the new Producer APIs, and set a producer config “transactional.id” to some unique ID. This unique ID is needed to provide continuity of transactional state across application restarts.

The real deal: Exactly-once stream processing in Apache Kafka

Building on idempotency and atomicity, exactly-once stream processing is now possible through the Streams API in Apache Kafka. All you need to make your Streams application employ exactly-once semantics, is to set this config “processing.guarantee=exactly_once”. This causes all of the processing to happen exactly once; this includes making both the processing and also all of the materialized state created by the processing job that is written back to Kafka, exactly once.

> *"This is why the exactly-once guarantees provided by Kafka's Streams API are the strongest guarantees offered by any stream processing system so far. It offers end-to-end exactly-once guarantees for a stream processing application that extends from the data read from Kafka, any state materialized to Kafka by the Streams app, to the final output written back to Kafka. Stream processing systems that only rely on external data systems to materialize state support weaker guarantees for exactly-once stream processing. Even when they use Kafka as a source for stream processing and need to recover from a failure, they can only rewind their Kafka offset to reconsume and reprocess messages, but cannot rollback the associated state in an external system, leading to incorrect results when the state update is not idempotent."*

Let me explain that in a little more detail. The critical question for a stream processing system is "does my stream processing application get the right answer, even if one of the instances crashes in the middle of processing?" The key, when recovering a failed instance, is to resume processing in exactly the same state as before the crash.

Now, stream processing is nothing but a read-process-write operation on a Kafka topic; a consumer reads messages from a Kafka topic, some processing logic transforms those messages or modifies state maintained by the processor, and a producer writes the resulting messages to another Kafka topic. Exactly-once stream processing is simply the ability to execute a read-process-write operation exactly one time. In this case, "getting the right answer" means not missing any input messages or producing any duplicate output. This is the behavior users expect from an exactly-once stream processor.

There are many other failure scenarios to consider besides the simple one we've discussed so far:

- The stream processor might take input from multiple source topics, and the ordering across these source topics is not deterministic across multiple runs. So if you re-run your stream processor that takes input from multiple source topics, it might produce different results.
- Likewise the stream processor might produce output to multiple destination topics. If the producer cannot do an atomic write across multiple topics, then the producer output can be incorrect if writes to some (but not all) partitions fail.
- The stream processor might aggregate or join data across multiple inputs using the managed state facilities the Streams API provides. If one of the instances of the stream processor fails, then you need to be able to rollback the state materialized by that instance of the stream processor. On restarting the instance, you also need to be able to resume processing and recreate its state.
- The stream processor might look up enriching information in an external database or by calling out to a service that is updated out of band. Depending on an external service makes the stream processor fundamentally non-deterministic; if the external service changes its internal state between two runs of the stream processor, it leads to incorrect results downstream. However, if handled correctly, this should not lead to entirely incorrect results. It should just lead to the stream processor output belonging to a set of *legal* outputs. More on this later in the blog.

Failure and restart, especially when combined with non-deterministic operations and changes to the persistent state computed by the application, may result not only in duplicates but in incorrect results. For example, if one stage of processing is computing a count of the number of events seen, then a duplicate in an upstream processing stage may lead to an incorrect count downstream. So we must qualify the phrase "exactly once stream processing." It refers to consuming from a topic, materializing intermediate state in a Kafka topic and producing to one, not all possible computations done on a message using the Streams API. Some computations (for example, depending on an external service or consuming from multiple source topics) are fundamentally non-deterministic.

> *"The correct way to think of exactly-once stream processing guarantees for deterministic operations is to ensure that the output of a read-process-write operation would be the same as it would if the stream processor saw each message exactly one time—as it would in a case where no failure occurred."*

Wait, but what's exactly-once for non-deterministic operations anyway?

That makes sense for deterministic operations, but what does exactly-once stream processing mean when the processing logic itself is non-deterministic? Let's say the same stream processor that keeps a running count of incoming events is modified to count only those events that satisfy a condition dictated by an external service. Fundamentally, this operation is non-deterministic in nature, since the external condition can change between two distinct runs of the stream processor, potentially leading to different results downstream. So what's the right way to think about exactly-once guarantees for non-deterministic operations like this?

> *"The correct way to think of exactly-once guarantees for non-deterministic operations is to ensure that the output of a read-process-write stream processing operation belongs to the subset of legal outputs that would be produced by the combinations of legal values of the non-deterministic input."*

So for our example stream processor above, for a current count of 31 and an input event value of 2, correct output under failures can only be one of {31, 33}: 31 if the input event is discarded as indicated by the external condition, and 33 if it is not.

This article only scratches the surface of exactly-once stream processing in the Streams API. An upcoming post on this topic will describe the guarantees in more detail as well as talk about how they compare to exactly-once guarantees in other stream processing systems.

Exactly-once guarantees in Kafka: Does it actually work?

With any major body of work like this one, a common question is whether the feature works as promised or not. To answer this question for the exactly-once guarantees in Kafka, let's look into correctness (that is, how we designed, built, and tested this feature) and performance.

A meticulous design and review process

Correctness and performance both start with a solid design. We started working on a design and prototyping about three years ago at LinkedIn. We iterated on this for over a year at Confluent looking for an elegant way to converge the idempotence and transactional requirements into a holistic package. We wrote a >60 page design document that outlined every aspect of the design; from the high-level message flow to the nitty-gritty implementation details of every data structure and RPC. This went through an extensive public scrutiny over a 9 month period in which the design improved substantially from community feedback. For instance, thanks to the open source discussion, we replaced consumer side buffering for transactional reads with smarter server side filtering, thus avoiding a potentially big performance overhead. In a similar vein, we also refined the interplay of transactions with compacted topics and added security features.

As a result, we ended up with a simple design that also relies on the robust Kafka primitives to a substantial degree. To wit:

1. Our transaction log is a Kafka topic and hence comes with the attendant durability guarantees.
2. Our newly introduced *transaction coordinator* (which manages the transaction state per producer) runs within the broker and naturally leverages Kafka's leader election algorithm to handle failover.
3. For stream processing applications built using Kafka's Streams API, we leverage the fact that the the source of truth for the state store and the input offsets are Kafka topics. Hence we can transparently fold this data into transactions which atomically write to multiple partitions, and thus provide the exactly once guarantee for streams across the read-process-write operations.

This simplicity, focus on leverage, and attention to detail meant that the design had more than a fair chance of resulting in an implementation that works well.

An iterative development process

We developed the feature in the open to ensure that every pull request went through an extensive review. That meant putting some of the pull requests through several dozen iterations over a period of months. This review process found some gaps in the design and innumerable corner cases which were not previously considered.

We wrote over 15,000 LOC of tests, including distributed tests running with real failures under load and ran them every night for several weeks looking for problems. This uncovered all manner of issues, ranging from basic coding mistakes to esoteric NTP synchronization issues in our test harness. A subset of these were distributed chaos tests, where we bring up a full Kafka cluster with multiple transactional clients, produce message transactionally, read these messages concurrently, and hard kill clients and servers during the process to ensure that data is neither lost nor duplicated.

As a result, a simple and solid design with a well-tested, high quality code base forms the bedrock of our solution.

The good news: Kafka is still fast!

While designing this feature, a key focus was performance; we wanted our users to be able to use exactly-once delivery and processing semantics beyond just a handful of niche use cases and be able to actually turn it on by default. We eliminated a lot of simpler design alternatives due to the performance overhead that came with those designs. After much thought, we settled on a design that involves minimal overhead per transaction (~1 write per partition and a few records appended to a central transaction log). This shows in the measured performance of this feature. For 1KB messages and transactions lasting 100ms, the producer throughput declines only by 3%, compared to the throughput of a producer configured for at least once, in order delivery (acks=all, max.in.flight.requests.per.connection=1), and by 20% compared to the throughput of a producer configured for most once delivery with no ordering guarantees (acks=1, max.in.flight.requests.per.connection=5), which is the current default. There are more performance improvements lined up after this first release of exactly-once semantics. For instance, once we check in KAFKA-5494 that improves pipelining in the producer, we expect the transactional producer throughput overhead to reduce significantly even when compared to a producer that supports at most once delivery with no ordering guarantees. We also found that idempotence has a negligible impact on producer throughput. If you are curious, we have published the results of our benchmarking, our test setup, and our test methodology.

In addition to ensuring low performance overhead for the new features, we also didn't want to see a performance regression in applications that didn't use the exactly-once features. To ensure that, we not only added some new fields in the Kafka message header to implement the exactly-once features, but we also reworked the Kafka message format to compress messages more effectively over the wire and on disk. In particular, we moved a bunch of common metadata data into batch headers, and introduced variable length encoding into each record within the batch. With this smart batching, the overall message size is significantly smaller. For instance, a batch of 7 records of 10 bytes each would be 35% smaller in the new format. This led to a net gain in raw Kafka performance for I/O bound applications—Up to a 20% improvement in producer throughput and up to a 50% improvement in consumer throughput while processing small messages. **This performance boost is available to any Kafka 0.11 user, even if you don't use any of the exactly-once features.**

We also had a look into the overhead of exactly-once stream processing using the Streams API. With a short commit interval of 100ms—that is required to keep end-to-end latency low—we see a throughput degradation of 15% to 30% depending on the message size; a 1KB message size for the former and 100 bytes for the latter. However, a larger commit interval of 30 sec has no overhead at all for larger message sizes of 1KB or higher. For the next release, we plan to introduce speculative execution that will allow us to keep end-to-end latency low even if we use a larger commit interval. Thus, we expect to get the overhead of transactions down to zero.

In summary, by fundamentally reworking some of our core data structures, we gave ourselves the headroom to build the idempotence and transactions feature for minimal performance overhead and to make Kafka faster for everyone. A lot of that hard work and years down the line, we are beyond excited to release the exactly-once feature for the broad Apache Kafka community. For all the diligence that went into building exactly-once semantics in Kafka, there are improvements that will follow as the feature gets widely adopted by the community. We look forward to hearing this feedback and iterating on improvements in upcoming releases of Apache Kafka.

Is this Magical Pixie Dust I can sprinkle on my application?

No, not quite. Exactly-once processing is an end-to-end guarantee and the application has to be designed to not violate the property as well. If you are using the consumer API, this means ensuring that you commit changes to your application state concordant with your offsets as described here.

For stream processing, the story is actually a bit better. Because stream processing is a closed system, where input, output, and state modifications are all modeled in the same operation, it actually is a bit like magic pixie dust. A single config change will give you the end-to-end guarantee. You still need to get the data out of Kafka, though. When combined with an exactly-once connector you'll have this property.

Additional Resources

If you'd like to understand the exactly-once guarantees in more detail, I'd recommend poring over KIP-98 for the transactions feature and KIP-129 for exactly-once stream processing. If you'd like to dive deeper into the design of these features, this design document is a great read.

This post primarily focussed on describing the nature of the user-facing guarantees as supported by the upcoming exactly-once capability in Apache Kafka 0.11, and how you can use the feature. In the following posts in this series, we will go into more details of the various aspects of exactly-once guarantees—idempotence, transactions and exactly-once stream processing. Exactly-once is now available in Confluent Platform 3.3 in both the open source and enterprise version.

Acknowledgements

An amazing team of distributed systems engineers worked for over a year to bring the exactly-once work to fruition: Jason Gustafson, Guozhang Wang, Apurva Mehta, Matthias Sax, Damian Guy, Eno Thereska, Sriram Subramanian, and Flavio Junqueira.